

Tasador SD

Auditoría integral del producto

Consolidación, auditoría técnica y de seguridad, sistema de diseño, política de seguridad, roadmap y checklists de lanzamiento de la plataforma SaaS de tasación de alquileres para Santo Domingo, República Dominicana.

Producto	Tasador SD - plataforma de tasación de alquileres
Repositorio principal	github.com/Criscarr26/tasador-sd
Cliente móvil (demo)	github.com/Criscarr26/rental-estimator-mobile
Versión del documento	1.1
Fecha	8 de julio de 2026
Preparado para	Cristian Carrera
Clasificación	Interno / presentable a inversionistas

Tabla de contenidos

Tabla de contenidos	2
1. Resumen ejecutivo	4
1.1 Veredicto	4
2. Consolidación del proyecto (Fase 1)	4
2.1 Inventario completo y decisión de consolidación	4
2.2 Duplicados e ideas descartadas eliminados	4
2.3 Versión oficial monetizable	5
3. Arquitectura final del sistema	5
3.1 Vista de componentes	5
3.2 Flujo de datos	5
3.3 Decisiones de arquitectura registradas	5
4. Tareas pendientes que requieren tu intervención (Fase 2)	6
4.1 Detalle de las tareas críticas y altas	6
4.2 Notas sobre el resto	8
5. Auditoría técnica (Fase 3)	8
5.1 Fortalezas verificadas	8
5.2 Problemas detectados, impacto y solución	8
5.3 Escalabilidad	9
5.4 Experiencia de desarrollo (DX)	9
6. Auditoría de seguridad (Fase 5)	9
6.1 Backend e inyecciones	9
6.2 Frontend	10
6.3 APIs, autenticación y autorización	10
6.4 Infraestructura	10
6.5 Base de datos	10
6.6 Datos personales y sensibles	10
6.7 Contraseñas y sesiones	10
6.8 Seguridad de IA (el agente)	11
6.9 Pagos (diseño preventivo obligatorio)	11
7. Diseño: crítica y sistema de diseño (Fase 4)	11
7.1 Crítica del estado actual	11
7.2 Sistema de diseño (tokens oficiales)	12
7.3 Inventario de componentes reutilizables	12
8. Recomendaciones de mejora priorizadas	13
9. Política de seguridad (Fase 6)	13

9.1 Objetivos y alcance	13
9.2 Roles y responsabilidades	13
9.3 Gestión de accesos, autenticación y autorización	13
9.4 Gestión de secretos	14
9.5 Desarrollo seguro (SDLC) y gestión de cambios	14
9.6 Protección de datos, privacidad y cifrado	14
9.7 Backups y recuperación ante desastres	14
9.8 Respuesta a incidentes	14
9.9 Registro, monitoreo y cumplimiento	14
9.10 Checklist de despliegue seguro (cada release)	15
10. Roadmap (Fase 7)	15
11. Checklist previo al lanzamiento (beta pública)	15
12. Checklist para producción (operación continua)	15
13. Riesgos técnicos y plan de mitigación	16
14. Anexo: alcance y método de esta auditoría	16

1. Resumen ejecutivo

Tasador SD es una plataforma SaaS que estima el precio mensual de alquiler de propiedades en Santo Domingo mediante un modelo de machine learning calibrado por sector. El producto nació como tres proyectos independientes (una web de demostración en Streamlit, un agente autónomo de recolección de datos y una aplicación móvil) y fue unificado en una sola arquitectura con un contrato de dominio único, una sola fuente de inferencia, identidad compartida entre clientes y un ciclo de datos diseñado para mejorar el modelo con datos reales del mercado.

El estado actual es sólido para una beta privada: el monorepo está publicado con integración continua en verde, el API de inferencia reproduce exactamente las predicciones del modelo entrenado (verificado con tests de contrato), la web y la móvil comparten cuenta e historial sobre Supabase con Row Level Security verificado en producción, y el endurecimiento básico (CORS con lista blanca, security headers, CSP sin violaciones) está aplicado y probado.

Las brechas que separan el estado actual de un lanzamiento comercial son conocidas y acotadas: aplicar las migraciones de base de datos, desplegar API y web en sus plataformas gratuitas, reactivar la confirmación de correo, sustituir el dataset sintético por datos reales (el agente ya está construido y en pausa por decisión de negocio), añadir claves de API para el acceso medido de terceros e integrar la pasarela de pagos. El endurecimiento de la superficie pública ya está aplicado y verificado (rate limiting por IP, CORS con lista blanca, security headers, contenedor no-root y constraints de base de datos). Este documento detalla cada tarea con su prioridad, esfuerzo y riesgo.

1.1 Veredicto

Apto para beta privada de inmediato; apto para comercialización tras completar las tareas críticas y altas de la sección 4 (estimado total: 2 a 3 semanas de trabajo a tiempo parcial). La base arquitectónica no requiere rehacerse para escalar a los primeros miles de usuarios.

2. Consolidación del proyecto (Fase 1)

2.1 Inventario completo y decisión de consolidación

Componente	Ubicación	Decisión
Monorepo tasador-sd (API FastAPI, web Next.js, core-py, agente, entrenamiento, migraciones)	github.com/Criscarr26/tasador-sd	PRODUCTO OFICIAL. Toda evolución ocurre aquí.
App móvil Expo (rental-estimator-mobile)	github.com/Criscarr26/rental-estimator-mobile	Cliente oficial móvil. Repo separado como demo pública; candidato a integrarse al monorepo en v1.1.
Web Streamlit (rental-price-estimator-sd)	rental-price-estimator-sd.streamlit.app	Se conserva SOLO como demo de marketing/portafolio. No recibe funcionalidades nuevas.
Agente standalone (rental-listings-agent)	github.com/Criscarr26/rental-listings-agent	OBSOLETO como línea de desarrollo: la copia del monorepo es la oficial (usa tasador-core y escribe a Postgres). El repo público queda como pieza de portafolio.
Copias locales en ai-portfolio	Documentos/Github repository/ai-portfolio	Originales intactos como archivo histórico; no editar.

2.2 Duplicados e ideas descartadas eliminados

- La lógica de predicción existía dos veces (pipeline .pkl y port TypeScript) y la lista de sectores tres veces; hoy tasador-core es la única definición y el port móvil se verifica contra casos de referencia en CI.

- Los rangos de validación divergentes (área 20-500 vs 20-1000 vs 15-1000; antigüedad 0-60 vs 0-80) se resolvieron con una decisión documentada de producto.
- El export manual de pesos hacia la móvil se eliminó: entrenar produce model_params.json en la misma corrida.
- Herramientas evaluadas y descartadas en chats anteriores: Rork (generador de apps de pago, misma tecnología Expo sin integración), graphify y agent-skills (no aportaban al flujo actual).
- Corotos como fuente del agente: descartado en v1 (renderiza con JavaScript); SuperCasas es la fuente v1.

2.3 Versión oficial monetizable

La versión monetizable es el monorepo tasador-sd + la app móvil, operando sobre un único proyecto Supabase: web comercial (Next.js) y móvil (Expo) como puntos de venta, API de inferencia como producto técnico (futuro plan API para terceros), y el agente + reentrenamiento como ventaja competitiva de datos. El modelo de negocio freemium ya tiene cimientos ejecutables: planes en la tabla perfiles y límite mensual del plan gratis aplicado por triggers en la base de datos.

3. Arquitectura final del sistema

3.1 Vista de componentes

Capa	Componente	Responsabilidad única
Dominio	packages/core_py (tasador-core)	Schema, sectores, rangos, validación, construcción/predicción del pipeline y export de pesos. Fuente única de verdad; 9 tests incluyendo el contrato de paridad.
Inferencia	apps/api (FastAPI)	POST /v1/appraisals (validación + tasación + rango de confianza) y GET /v1/model/params (pesos versionados por hash de contenido). Única fuente de inferencia servida.
Cliente web	apps/web (Next.js 16)	Landing comercial + tasador (consume el API, nunca reimplementa el modelo) + historial.
Cliente móvil	rental-estimator-mobile (Expo SDK 54)	Captura en campo con predicción on-device (pesos sincronizados del API, fallback embebido) e historial automático.
Identidad y datos	Supabase (Auth + Postgres + RLS)	Cuenta compartida web/móvil; saved_estimates (historial), listings (datos del agente), perfiles/usage_counters (planes y límites). RLS en todas las tablas.
Datos	agents/listings-agent + ml/training	Recolección responsable de listados reales (robots.txt, rate limit, presupuestos) hacia Postgres; reentrenamiento reproducible que propaga artefactos a todos los clientes.
Operación	CI GitHub Actions + Dockerfile + docs/DEPLOY.md	Tests de contrato y build web en cada push; contenedor del API listo para HF Spaces.

3.2 Flujo de datos

- Tasación: cliente -> POST /v1/appraisals -> validación (tasador-core) -> pipeline .pkl -> respuesta con estimado, rango +/- RMSE, promedio del sector y versión del modelo.
- Historial: cliente autenticado -> insert en saved_estimates -> trigger de límites (0003) cuenta el uso -> RLS garantiza que solo el dueño lee/borra sus filas.
- Ciclo de datos: agente -> listings (upsert por URL, service role) -> reentrenamiento lee listings -> nuevos artefactos versionados -> API los sirve -> la móvil sincroniza pesos y la web consume el API.
- Sincronía de modelo: la versión es un hash del contenido de los pesos; reentrenar cambia la versión y los clientes la detectan sin intervención manual.

3.3 Decisiones de arquitectura registradas

- Sin Redis ni colas dedicadas en esta escala: caché en memoria del API y GitHub Actions cron como scheduler del agente. Los puntos de inserción quedan definidos para cuando el tráfico lo exija.
- Multi-tenant preparado, no construido: columna org_id se añadirá cuando exista el plan Agencia; el tenant natural del mercado dominicano es la inmobiliaria con varios agentes.
- Streamlit se mantiene como demo por costo cero de mantenimiento y valor de marketing.
- Fuentes del sistema en web y móvil: los CDN de fuentes se cuelgan en redes con inspección TLS (lección operativa del entorno de desarrollo).

4. Tareas pendientes que requieren tu intervención (Fase 2)

Consolidado de TODO lo que en este chat y en los anteriores quedó señalado como acción tuya. La tabla resume; debajo, cada tarea tiene su detalle completo (motivo, pasos exactos, dependencias y riesgo de no hacerla). Fechas recomendadas contadas desde el 8 de julio de 2026.

#	Prioridad	Tarea	Tiempo	Fecha recomendada	Estado
1	Crítica	Aplicar migraciones 0002 y 0003 en Supabase	10 min	Esta semana	Pendiente
2	Crítica	Desplegar el API (Hugging Face Spaces)	30-45 min	Esta semana	Pendiente
3	Crítica	Desplegar la web (Vercel)	20 min	Esta semana	Pendiente
4	Crítica	Eliminar cuentas de prueba de Supabase	3 min	Hoy	Pendiente
5	Alta	Reactivar confirmación de correo (Confirm email)	2 min	Antes de usuarios reales	Pendiente
6	Alta	Apuntar la móvil al API desplegado	5 min	Tras tarea 2	Pendiente
7	Alta	Decidir plan de monetización y autorizar corrida del agente	Decisión + ~US\$5	2 semanas	En pausa (tu decisión)
8	Alta	Reentrenar con datos reales y publicar	30 min	Tras tarea 7	Bloqueada por 7
9	Alta	Crear API key de Anthropic para el agente	10 min	Con tarea 7	Pendiente (pediste que te lo recuerde)
10	Media	Sentry + monitoreo de disponibilidad	45 min	Semana 2	Pendiente
11	Media	Dominio propio + actualizar CORS/CSP	30 min + US\$10-15/año	Semana 3	Pendiente
12	Media	Pasarela de pagos (Stripe; evaluar Azul/CardNet)	1-2 días	Mes 2	Pendiente
13	Media	Distribución iOS real (EAS/TestFlight)	2-3 h + US\$99/año	Cuando haya clientes móviles	Pendiente
14	Media	Política de privacidad y términos de uso	2-3 h	Antes de beta pública	Pendiente
15	Baja	Dependabot + npm audit en los repos	20 min	Semana 3	Pendiente
16	Baja	Verificar backups de Supabase y export mensual	15 min	Semana 3	Pendiente
17	Baja	Aplanar carpetas 'Otros' del portafolio (pendiente de chats previos)	30 min	Sin fecha	Pendiente

4.1 Detalle de las tareas críticas y altas

Tarea 1 - Aplicar migraciones 0002 y 0003

Motivo	Sin 0002 no existe la tabla listings (el agente no tiene destino); sin 0003 no hay planes, contadores de uso ni límite del plan gratis. El RLS de esas tablas viene dentro de las migraciones.
--------	--

Qué hacer	Supabase -> SQL Editor (página SIN traducir por Chrome) -> pegar y ejecutar supabase/migrations/0002_listings.sql -> Run -> repetir con 0003_plans_usage.sql. Avísame al terminar y verifico el RLS de las tablas nuevas por REST.
Dependencias	Ninguna.
Riesgo de no hacerlo	El freemium no se puede activar; corrida del agente bloqueada; cualquier usuario podría acumular tasaciones sin límite.

Tarea 2 - Desplegar el API

Motivo	Sin API público la web desplegada no puede tasar y la móvil no sincroniza pesos.
Qué hacer	Crear cuenta en huggingface.co -> New Space -> SDK Docker -> subir el repo (el Dockerfile de la raíz ya sirve en el puerto 7860) -> en Variables definir ALLOWED_ORIGINS=https://TU-WEB.vercel.app -> verificar /health. Guía completa: docs/DEPLOY.md.
Dependencias	Cuenta de Hugging Face (gratis).
Riesgo de no hacerlo	El producto solo funciona en tu máquina.

Tarea 3 - Desplegar la web

Motivo	Es la cara comercial del SaaS.
Qué hacer	vercel.com -> New Project -> importar Criscarr26/tasador-sd -> Root Directory: apps/web (crítico) -> variables NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY -> Deploy.
Dependencias	Tarea 2 (para la URL del API).
Riesgo de no hacerlo	Sin canal de adquisición ni demo comercial pública.

Tarea 4 - Eliminar cuentas de prueba

Motivo	Existen prueba.tasador.sd@gmail.com y prueba2.tasador.sd@gmail.com con contraseña conocida (aparece en nuestros chats). Riesgo bajo con RLS, pero es higiene básica.
Qué hacer	Supabase -> Authentication -> Users -> eliminar ambas cuentas (sus filas en saved_estimates se borran en cascada).
Dependencias	Ninguna.
Riesgo de no hacerlo	Cuentas activas con credenciales expuestas en conversaciones.

Tarea 5 - Reactivar Confirm email

Motivo	La desactivamos para probar rápido. En producción permite registrar correos ajenos.
Qué hacer	Supabase -> Authentication -> Sign In / Up -> Email -> activar Confirm email -> Save.
Dependencias	Hacerla justo antes de abrir el registro a usuarios reales.
Riesgo de no hacerlo	Suplantación de correos, spam de cuentas, reputación de dominio dañada.

Tarea 6 - Apuntar la móvil al API

Motivo	Activa la sincronización de pesos versionados (hoy usa el fallback embebido).
Qué hacer	En el .env de rental-estimator-mobile: EXPO_PUBLIC_MODEL_API_URL=https://TU-SPACE.hf.space -> reiniciar npx expo start.
Dependencias	Tarea 2.
Riesgo de no hacerlo	Tras un reentrenamiento la móvil quedaría con el modelo viejo hasta publicar build.

Tareas 7-9 - Monetización, datos reales y API key

Motivo	El modelo actual se entrena con 500 filas sintéticas: es honesto para demo, insuficiente para cobrar. El agente está construido y validado; su corrida en vivo (~US\$5) quedó EN PAUSA por tu decisión, ligada al chat de monetización que abriste (task_5348ed05). Pediste explícitamente que te recordara la creación de la API key de Anthropic antes de la corrida.
Qué hacer	Cerrar el plan de recuperación del gasto en ese chat -> crear la key en console.anthropic.com (Settings -> API Keys) y ponerla en agents/listings-agent/.env junto a SUPABASE_SERVICE_KEY -> python agent.py --site supercasas --target 20 --sink supabase -> reentrenar (python ml/training/train.py) -> commit de artefactos -> la versión del modelo cambia sola para todos los clientes.
Dependencias	Tareas 1 y 2; tu decisión de negocio.
Riesgo de no hacerlo	Vender precios calibrados sintéticamente: riesgo reputacional y de precisión.

4.2 Notas sobre el resto

- Sentry (tarea 10): crear cuenta gratis, un proyecto para web y otro para móvil; el API puede empezar con los logs del Space y UptimeRobot golpeando /health cada 5 minutos.
- Pagos (tarea 12): Stripe funciona en USD para RD; para tarjetas locales en DOP evaluar Azul o CardNet (requieren RNC y cuenta bancaria empresarial: ese trámite es tuyo). El diseño anti-fraude está en la sección 6.9.
- iOS (tarea 13): Expo Go de iOS está congelado en SDK 54 por Apple; para clientes reales usar EAS Build + TestFlight con cuenta Apple Developer.
- Privacidad (tarea 14): almacenar correos de usuarios; la beta pública necesita política de privacidad y términos visibles en la web.

5. Auditoría técnica (Fase 3)

5.1 Fortalezas verificadas

- Contrato de dominio único con test de paridad: es imposible que un cliente prediga distinto al modelo sin que CI falle. Es el hallazgo de mayor calidad del proyecto.
- Verificación de extremo a extremo real: login, tasación del caso de referencia (RD\$ 83,862), autoguardado e historial compartido se probaron contra servicios vivos, no solo con mocks.
- Reproducibilidad: entrenamiento con semilla fija reproduce métricas exactas (MAE 3,983 / RMSE 6,749 / R2 0.928).
- Seguridad de datos en el lugar correcto: RLS en Postgres (verificado: un anónimo ve 0 filas), no en los clientes.
- Costo de infraestructura actual: US\$0/mes (tiers gratuitos en todo).

5.2 Problemas detectados, impacto y solución

Problema	Por qué importa / impacto	Solución propuesta	Prioridad
El tasador público del API es anónimo (sin cuenta)	Es intencional: la tasación de la landing es el funnel de captación y debe funcionar sin login. El riesgo de abuso de cómputo YA está acotado por el rate limit por IP aplicado (60/min, configurable). Falta el acceso medido para terceros de pago.	Hecho: rate limit por IP. Pendiente (beta pública): tabla api_keys con hash para integraciones de terceros y conteo de tasaciones por clave; el /v1 ya versiona para evolucionar sin romper.	Media (rate limit ya mitiga; API keys antes de vender acceso)
Modelo entrenado con datos sintéticos	Precisión real desconocida frente al mercado; riesgo reputacional al cobrar.	Ejecutar el flywheel ya construido (tareas 7-9) y re-evaluar métricas con hold-out real.	Alta

Problema	Por qué importa / impacto	Solución propuesta	Prioridad
Observabilidad parcial (falta agregación de errores y uptime)	El API ya emite logging estructurado por request (método, ruta, estado, latencia); falta captura de errores de cliente y monitoreo de disponibilidad para enterarse de un fallo sin que lo reporte un usuario.	Añadir Sentry (web+móvil) y UptimeRobot en /health; el logging JSON del API ya está.	Media
El límite de uso vive en el guardado, no en la tasación	El trigger de 0003 cuenta inserts en saved_estimates; tasaciones anónimas via API no descuentan.	Al implementar auth en el API, contar también las tasaciones (tabla usage por API) y unificar la métrica de consumo.	Media (junto con auth del API)
Cold starts en tiers gratuitos	HF Spaces/Render duermen; la primera tasación tras inactividad puede tardar 20-60 s.	Ping periódico (UptimeRobot ya lo hace de gratis) o upgrade de tier cuando haya ingresos.	Baja
Dependencias con avisos (14 vulnerabilidades npm moderadas, pip/setuptools viejos en venvs)	Deuda de mantenimiento; mayormente cadena de desarrollo, no runtime.	Dependabot en ambos repos + npm audit fix en ventana controlada + actualizar pip/setuptools.	Baja
El .pkl exige la versión exacta de scikit-learn	Cargar el pipeline con otra major puede fallar (ya fijado en CI/Docker, pero es una restricción real).	Mantener el pin 1.9.0 y re-serializar el modelo en cada upgrade planificado de sklearn.	Baja (documentada)

5.3 Escalabilidad

El API es stateless (el modelo carga en memoria, ~1 ms por predicción de regresión lineal): escala horizontal trivial detrás de un balanceador. Supabase free soporta la beta; el primer cuello real será el pool de conexiones de Postgres alrededor de decenas de miles de usuarios activos, resoluble con el tier Pro y PgBouncer (incluido). La web es estática/SSR en CDN de Vercel. Con miles de usuarios no hay cambios arquitectónicos; con millones, separar lectura de historial (réplicas) y mover el rate limiting a un edge/WAF. Nada del diseño actual lo impide.

5.4 Experiencia de desarrollo (DX)

- Monorepo con verificación en un comando por pieza (unittest, npm run build, --dry-run) y CI en cada push.
- Entorno Windows con particularidades documentadas en la memoria del proyecto: inspección TLS (pip --trusted-host, truststore en el agente), .pth para instalar core_py en venvs con setuptools viejo, y Metro como servidor para capturas.
- Deuda menor: los venvs comparten propósitos (rental-sd aloja el API); un venv dedicado por app cuando el equipo crezca.

6. Auditoría de seguridad (Fase 5)

Enfoque de pentest de caja blanca sobre el código y la configuración reales, más diseño preventivo para los componentes aún no construidos (pagos). Estado por dominio y hallazgos accionables.

6.1 Backend e inyecciones

- **SQL/NoSQL injection: mitigado.** No hay SQL crudo en el producto; los clientes usan PostgREST (parametrizado) y el API no toca la base de datos. Las migraciones son DDL estático. Como defensa en profundidad, la migración 0004 añade CHECK constraints de valores en saved_estimates: aunque un cliente manipulado escriba en sus propias filas (RLS lo confina a ellas), no puede insertar valores fuera del dominio.
- **RCE / deserialización: bajo riesgo.** El único artefacto deserializado es el .pkl propio, generado por el pipeline de entrenamiento del repositorio; nunca cargar modelos de origen externo sin firma.

- **SSRF/XXE/Path traversal/File upload/Command injection: no aplican** en el API (no hay fetch de URLs del usuario, ni XML, ni rutas del usuario, ni uploads, ni shell). El único componente que visita URLs es el agente, tratado en 6.8.

6.2 Frontend

- **XSS: bajo.** React escapa por defecto y no se usa dangerouslySetInnerHTML. Residual: la CSP permite 'unsafe-inline' en scripts (requisito de Next sin nonces). Mejora recomendada: nonces CSP via middleware de Next (prioridad media).
- **Clickjacking: mitigado** (frame-ancestors 'none' + X-Frame-Options DENY, verificados en vivo).
- **CSRF: no aplica** al no usar cookies de sesión: los tokens viven en storage y viajan como Bearer. Trade-off consciente: el token es alcanzable por XSS, de ahí la CSP y la disciplina de dependencias.
- **CSP: activa y verificada sin violaciones** con connect-src limitado exactamente a Supabase y al API.

6.3 APIs, autenticación y autorización

- Supabase Auth emite JWT firmados; la autorización de datos es RLS a nivel de fila (verificada en producción para saved_estimates; listings/profiles/usage llegan con las migraciones pendientes).
- **Estado:** el rate limiting por IP ya está aplicado y probado (test que verifica el 429 con Retry-After). El tasador público es anónimo por diseño (funnel de captación); el guardado en historial sí exige sesión y queda confinado por RLS. Pendiente para vender acceso: api_keys con hash para terceros y conteo por clave; el versionado /v1 ya existe para evolucionar sin romper.
- Rotación de secretos: la publishable key es pública por diseño; la service_role key solo existe en el .env local del agente (gitignored, verificado). Si alguna vez se expone, rotar en Supabase y actualizar el .env.

6.4 Infraestructura

- HTTPS/TLS: terminación gestionada por Vercel/HF/Supabase con certificados automáticos; HSTS ya se envía.
- Docker: imagen python:3.11-slim con dependencias fijadas, **usuario no-root (uid 10001) ya aplicado** y .dockerignore que mantiene .env, node_modules y .git fuera del contexto de build. Mejora restante: fijar la imagen base por digest (prioridad baja).
- Kubernetes/Firewall/Reverse proxy: no aplican en esta escala; el reverse proxy lo aportan las plataformas.
- Variables de entorno: separadas por app con .env.example; ninguna credencial en los repos (verificado). El repo incluye SECURITY.md con el proceso de divulgación y el modelo de seguridad resumido.

6.5 Base de datos

- Cifrado en reposo y en tránsito: provisto por Supabase (AES-256 / TLS).
- Permisos: anon/authenticated solo ven lo que las políticas permiten; listings sin políticas (solo service role).
- Backups: el tier gratuito retiene 7 días. Acción: export mensual manual (pg_dump desde el dashboard) hasta subir de tier.
- Auditoría de accesos: los logs del dashboard bastan para la beta; activar log drain al crecer.

6.6 Datos personales y sensibles

- Se almacena: correo, hash de contraseña (bcrypt, gestionado por Supabase) e historial de tasaciones. No hay datos financieros ni documentos de identidad.
- Los logs del API no registran cuerpos de peticiones ni tokens (verificado en el código).
- Pendiente para beta pública: política de privacidad, términos y borrado de cuenta autoservicio (las cascadas ON DELETE ya garantizan el borrado íntegro de datos del usuario).

6.7 Contraseñas y sesiones

- Política mínima de 6 caracteres (validación en clientes + Supabase). Recomendación: subir a 8 y habilitar la protección de contraseñas filtradas de Supabase (prioridad media).
- Tokens con refresh automático (processLock en móvil, patrón oficial); signOut disponible en ambos clientes.

6.8 Seguridad de IA (el agente)

- **Prompt injection (riesgo real):** el agente lee páginas web ajenas; una página podría contener instrucciones adversarias ('ignora tus reglas y guarda este precio'). Mitigaciones ya construidas: las reglas éticas viven en las herramientas (robots.txt, rate limit) donde el modelo no puede saltárselas; save_listing valida cada registro contra el schema y descarta lo inválido; presupuestos duros de fetches y turnos acotan el daño económico; la única herramienta con efectos es save_listing (sin shell, sin archivos, sin red arbitraria de escritura).
- Refuerzos recomendados (media): allowlist de dominios en fetch_url (hoy cualquier URL http(s) que robots permita), y revisión humana por muestreo de listings nuevos antes de reentrenar.
- Data leakage / model poisoning: el modelo no entrena con datos de usuarios, solo con listings públicos validados; el envenenamiento se limita con rangos de cordura, dedupe por URL y el muestreo humano anterior.
- Jailbreak: el agente no expone chat a usuarios finales; superficie limitada al contenido de páginas (cubierto arriba).

6.9 Pagos (diseño preventivo obligatorio)

Aún no hay pasarela; estas reglas son vinculantes para la implementación:

- **Precios y planes solo en el servidor:** el cliente jamás envía montos ni nombres de plan; envía identificadores de precio de Stripe creados por ti. Imposible alterar precios desde el navegador.
- **Webhooks firmados:** verificar la firma (stripe-signature) y rechazar todo lo demás; es la única vía que cambia profiles.plan. Falsificar un webhook sin la clave de firma resulta imposible.
- **Idempotencia:** guardar event.id procesados; un webhook duplicado (o reintentado por un atacante) no duplica créditos ni renovaciones.
- **Condiciones de carrera:** el patrón ya existe en el código: contadores con SELECT ... FOR UPDATE y unicidad por período. Aplicar el mismo patrón a créditos de pago.
- **Acceso premium sin pagar:** el plan vive en profiles (RLS: el usuario NO puede escribir su propia fila de plan; solo el service role via webhook). Los límites se aplican por triggers en la base, no en el cliente.
- **Promociones:** cupones de un solo uso ligados a user_id con unique constraint; nunca códigos acumulables sin registro.
- **Reembolsos/chargebacks:** webhook de disputa degrada el plan automáticamente y registra el evento.

7. Diseño: crítica y sistema de diseño (Fase 4)

7.1 Crítica del estado actual

Lo que ya está a nivel de producto: identidad coherente web/móvil (navy #0B1121 + acento cian-índigo), jerarquía tipográfica clara, tarjetas con elevación consistente, estados de carga (skeletons), vacíos y de error en los flujos principales, y microinteracciones básicas (hover con elevación, transiciones de barras, botón con gradiente y estado presionado).

Lo que falta para parecer una empresa grande:

Área	Brecha	Acción de rediseño	Prioridad
Landing	Vende la herramienta, no el negocio: falta página de precios, testimonios/casos, FAQ y CTA de registro directo.	Añadir secciones Precios (3 planes), Cómo funciona (3 pasos), FAQ y footer con legal. El tasador pasa a ser la demo dentro del funnel.	Alta (pre-beta pública)
Formularios	Validación solo al enviar en la web; sin mensajes por campo.	Validación inline con mensajes bajo cada campo y aria-invalid; deshabilitar submit solo con errores.	Media
Navegación	Dos páginas planas; sin dashboard.	App shell con sidebar (Tasador, Historial, Estadísticas, Cuenta, Plan) cuando exista el dashboard de métricas del usuario.	Media (v1.0)
Accesibilidad	Contraste del texto muted (#8FA3C4 sobre navy) roza el mínimo AA en tamaños pequeños; focus visible depende del default del navegador.	Subir muted un paso (#9FB3D4) para texto <14 px y definir focus ring propio (outline 2px accent + offset).	Media
Microinteracciones	Sin transiciones de página ni feedback de éxito más allá de texto.	Toasts de confirmación, transiciones de entrada (100-150 ms, respetando prefers-reduced-motion) y contador animado en el precio del resultado.	Baja
Móvil	El stepper y el bottom sheet están bien; falta haptic feedback y pull-to-refresh en Historial.	expo-haptics en acciones primarias; RefreshControl en la lista.	Baja

7.2 Sistema de diseño (tokens oficiales)

Formalización de lo ya construido; estos valores son la referencia para web (CSS variables), móvil (constants/theme.ts) y cualquier pieza futura:

Token	Valor	Uso
--bg / background	#0B1121	Fondo base (dark-first)
--card / card	#141D33	Superficies elevadas
--card-2 / cardSelected	#1C2A4A	Superficie seleccionada/hover
--border	#243252 (+ variante suave 14% alpha)	Bordes y divisores
--text	#F1F5F9	Texto principal
--muted	#8FA3C4 (subir a #9FB3D4 en texto pequeño)	Texto secundario
--accent / gradiente	#38BDF8 -> #818CF8 (135°)	CTAs, resaltados, marca
--success / --danger	#34D399 / #FB7185	Estados
Tipografía	Pila del sistema (Segoe UI Variable / SF Pro / Roboto)	Sin CDNs externos (decisión operativa)
Escala tipográfica	12 / 13.5 / 15 / 17 / 21 / 26 / 32 / 40 px (ratio ~1.25)	Jerarquía completa
Espaciado	4 / 8 / 16 / 24 / 32 / 64 px	Escala única web+móvil
Radios	10 (inputs) / 12 (botones) / 14 (controles) / 18-24 (tarjetas/sheets)	Elevación por redondeo
Sombra de tarjeta	0 18px 44px rgba(2,6,23,0.38)	Única elevación; no apilar sombras
Estados de componente	default / hover / active / focus-visible / disabled / loading / error / empty	Todo componente nuevo define los 8

7.3 Inventario de componentes reutilizables

- Existentes: Card, botón primario (gradiente) y secundario, Input/Select/Stepper, SectorPicker (bottom sheet móvil / select web), Skeleton, EmptyState, banner de error, tarjeta de historial, barras de mercado, Header con sesión.

- Por crear en v1.0: Toast, Modal de confirmación (sustituye el doble-tap de borrar), Badge de plan, Tabla de precios, Sidebar/AppShell, gráfica de uso del dashboard.
- Iconografía: Ionicons en móvil; adoptar Lucide en web (mismo estilo de trazo, tree-shakeable) con tamaño base 18/20 px y stroke 1.5.

8. Recomendaciones de mejora priorizadas

Prioridad	Recomendación	Resultado esperado
1 - Crítica	Completar tareas 1-4 de la sección 4 (migraciones, deploys, higiene de cuentas).	Producto vivo en internet con RLS completo.
2 - Alta	Claves de API (api_keys con hash) para acceso medido de terceros y conteo de tasaciones unificado. (El rate limiting por IP ya está aplicado.)	El freemium y el acceso B2B a la API se vuelven exigibles.
3 - Alta	Ejecutar el flywheel de datos reales (agente -> reentrenar -> publicar).	Precisión de mercado real; argumento de venta honesto.
4 - Alta	Landing de negocio (precios, cómo funciona, FAQ) + política de privacidad.	Funnel de conversión y cumplimiento mínimo para beta pública.
5 - Media	Sentry + UptimeRobot + logging estructurado.	Fallos visibles antes de que los reporte un cliente.
6 - Media	Pasarela de pagos con las reglas de 6.9.	Ingresos con superficie de fraude minimizada.
7 - Media	Nonces CSP, focus ring propio, contraste AA, validación inline.	Calidad de empresa grande en detalles perceptibles.
8 - Baja	Dependabot, digest pinning de la imagen base, export mensual de BD, EAS/TestFlight.	Higiene operativa sostenida.

9. Política de seguridad (Fase 6)

Política operativa de Tasador SD, versión 1.1 (julio 2026). Aplica a todo el código, datos e infraestructura del producto. El repositorio incluye una copia operativa en SECURITY.md con el proceso de divulgación. Hoy todos los roles los ejerce el fundador; cada responsabilidad se delega explícitamente cuando el equipo crezca.

9.1 Objetivos y alcance

- Proteger los datos de los usuarios (correo, credenciales, historial de tasaciones) y la integridad del modelo y sus datos de entrenamiento.
- Garantizar disponibilidad razonable del servicio y capacidad de recuperación ante incidentes.
- Alcance: monorepo tasador-sd, repo móvil, proyecto Supabase, despliegues (HF Spaces, Vercel, Expo), cuentas de plataformas y secretos asociados.

9.2 Roles y responsabilidades

- Propietario del producto y de seguridad: Cristian Carrera (aprueba cambios de esquema, secretos, despliegues y respuesta a incidentes).
- Todo colaborador futuro firma estas normas antes de recibir acceso; los accesos se otorgan por rol y con el mínimo privilegio.

9.3 Gestión de accesos, autenticación y autorización

- Cuentas de plataforma (GitHub, Supabase, Vercel, HF, Anthropic) con contraseña única gestionada en un gestor de contraseñas y 2FA activado (acción inmediata si falta en alguna).

- Usuarios finales: Supabase Auth con confirmación de correo en producción; contraseñas mínimo 8 caracteres y chequeo de filtraciones activado.
- Autorización de datos exclusivamente por RLS en Postgres; ninguna regla de negocio de acceso en clientes.
- El API validará JWT de Supabase para operaciones de usuario y api_keys con hash para integraciones.

9.4 Gestión de secretos

- Prohibido commitar secretos; .env gitignored en todos los repos (verificado). Los ejemplos usan placeholders.
- La service_role key solo existe en el .env local del agente y en variables de plataforma; nunca en clientes ni en logs.
- Rotación: inmediata ante sospecha de exposición; programada anual para claves de larga vida.

9.5 Desarrollo seguro (SDLC) y gestión de cambios

- Todo cambio pasa por git con CI en verde (tests de contrato + build) antes de desplegar.
- Cambios de esquema solo mediante archivos en supabase/migrations, numerados y aplicados en orden.
- Revisión obligatoria (humana o asistida) para cambios en auth, RLS, pagos o el agente.
- Dependencias: Dependabot activo; npm audit y pip list --outdated revisados mensualmente; actualizaciones de seguridad en menos de 7 días.

9.6 Protección de datos, privacidad y cifrado

- Datos personales limitados al mínimo (correo e historial); sin datos financieros propios (la pasarela los custodia).
- Cifrado en tránsito (TLS en todos los orígenes; HSTS activo) y en reposo (gestionado por Supabase).
- Derechos del usuario: borrado de cuenta elimina sus datos por cascada; política de privacidad pública antes de la beta abierta.
- Los logs nunca contienen tokens, contraseñas ni cuerpos de peticiones con datos personales.

9.7 Backups y recuperación ante desastres

- Supabase: backups automáticos (7 días en tier actual) + export manual mensual archivado localmente fuera de OneDrive.
- Código: GitHub es la fuente; los artefactos del modelo están versionados en el repo (recuperación total con git clone + DEPLOY.md).
- RTO objetivo de beta: 24 h; RPO: 24 h. Simulacro de restauración una vez por trimestre.

9.8 Respuesta a incidentes

- Detección: Sentry/UptimeRobot/logs. Clasificar en 1 h: ¿hay datos comprometidos?
- Contención: rotar claves afectadas, pausar despliegues, deshabilitar registro si procede.
- Erradicación y recuperación: parche con test que cubra la causa raíz; restaurar desde backup si hay corrupción.
- Comunicación: si hay datos personales afectados, notificar a los usuarios en 72 h con alcance y medidas.
- Post-mortem sin culpas en 7 días con acciones preventivas fechadas.

9.9 Registro, monitoreo y cumplimiento

- Uptime en /health cada 5 min; errores de cliente y servidor centralizados; revisión semanal de logs de auth (registros anómalos).

- Cumplimiento: producto dirigido a RD (Ley 172-13 de protección de datos personales como referencia); si se sirve a usuarios de la UE, evaluar GDPR antes de aceptar esos registros.

9.10 Checklist de despliegue seguro (cada release)

- CI en verde (contratos + build) y npm audit sin críticas nuevas.
- Sin secretos nuevos en el diff (git diff --stat + búsqueda de patrones de claves).
- Migraciones aplicadas en orden y RLS verificado si hubo cambios de esquema.
- Variables de entorno de producción confirmadas (ALLOWED_ORIGINS, NEXT_PUBLIC_*, MODEL_DIR).
- Prueba de humo post-deploy: /health, una tasación, un login y una lectura de historial.
- Sentry sin errores nuevos en los primeros 30 minutos.

10. Roadmap (Fase 7)

Etapas	Contenido	Duración estimada	Criterio de salida
MVP (completado)	Unificación, API de inferencia, web y móvil con auth e historial compartidos, límites en BD, CI, endurecimiento básico.	Hecho (jul 2026)	Todo verificado E2E - cumplido.
Beta privada	Tareas críticas 1-4; Sentry+uptime; 5-10 agentes inmobiliarios invitados; iteración de feedback semanal.	2 semanas	10 usuarios activos y 0 errores críticos en 2 semanas.
Beta pública	Datos reales (flywheel), api_keys para terceros, Confirm email, landing de negocio, privacidad/términos, dominio propio.	4-6 semanas	Registro abierto y métricas de retención recogándose.
Versión 1.0	Pagos (Stripe; evaluar Azul), plan Pro exigible, exportación PDF de tasaciones, dashboard de usuario, TestFlight iOS.	8-12 semanas	Primer cliente de pago.
Futuro (1.x-2.0)	Multi-tenant Agencia (org_id), API pública con claves para terceros, más ciudades (Santiago), app en tiendas, comparables por foto (visión).	Continuo	Decidir por tracción, no por calendario.

11. Checklist previo al lanzamiento (beta pública)

- ☐ Migraciones 0002 y 0003 aplicadas y RLS verificado por sonda externa.
- ☐ API desplegado con ALLOWED_ORIGINS correcto; /health monitoreado.
- ☐ Web en Vercel con las 3 variables y headers de seguridad servidos (verificar con curl -I).
- ☐ Móvil apuntando al API público; sincronización de pesos comprobada.
- ☐ Cuentas de prueba eliminadas; Confirm email activado; contraseña mínima 8.
- ☒ Rate limiting activo en el API (hecho). ☐ Claves de API para terceros si se vende acceso B2B.
- ☐ Modelo reentrenado con datos reales y métricas publicadas honestas en la landing.
- ☐ Política de privacidad y términos publicados y enlazados en registro.
- ☐ Sentry recibiendo eventos de web y móvil; UptimeRobot activo.
- ☐ Prueba E2E completa en producción (login, tasación, historial, borrado).
- ☐ Export de base de datos archivado (backup punto-cero del lanzamiento).

12. Checklist para producción (operación continua)

- ☐ Semanal: revisar Sentry, uptime, logs de auth y consumo de tiers gratuitos.

- [] Mensual: export de BD; npm audit / pip outdated; revisar costo si algún tier se acerca al límite.
- [] Por release: checklist de despliegue seguro (sección 9.10) completo.
- [] Trimestral: simulacro de restauración de backup; rotación de claves de larga vida si procede; revisión de esta política.
- [] Tras cada reentrenamiento: verificar versión nueva en /health, contraste de métricas y muestreo humano de listings usados.
- [] Ante incidente: aplicar 9.8 y registrar post-mortem en docs/.

13. Riesgos técnicos y plan de mitigación

Riesgo	Prob.	Impacto	Mitigación
Cambio de HTML/robots en SuperCasas rompe la recolección	Media	Sin datos frescos para reentrenar	El agente lee texto (no selectores frágiles); alertar si una corrida guarda 0 listings; sitemap de Corotos como fuente v2.
Apple mantiene congelado Expo Go (distribución iOS)	Alta	Demo iOS limitada a SDK 54	Ya mitigado fijando SDK 54; para clientes reales, EAS Build + TestFlight (tarea 13).
Abuso del API público de tasación	Baja	Costos/caída del tier gratuito	Mitigado: rate limit por IP (60/min) + ALLOWED_ORIGINS. Residual bajo hasta introducir acceso B2B medido con api_keys.
Deriva de versión de scikit-learn frente al .pkl	Baja	API no arranca tras upgrade descuidado	Versión fijada en CI/Docker/requirements; re-serializar el modelo en upgrades planificados.
Dependencia de tiers gratuitos (sleep, límites)	Media	Latencia percibida / interrupciones	Ping de uptime; presupuesto de upgrade definido (~US\$45/mes cubre Supabase Pro + hosting) al primer ingreso recurrente.
Pérdida del entorno local (única máquina de desarrollo)	Baja	Retraso de días	Todo reproducible desde GitHub + DEPLOY.md; secretos re-emitibles; backups de BD.
Datos sintéticos confundidos con reales por un cliente	Media	Reputacional	El disclaimer ya existe en web y demo; retirarlo únicamente tras el reentrenamiento real.

14. Anexo: alcance y método de esta auditoría

- Fuentes: código completo de ambos repositorios, historial de decisiones y pendientes registrados en la memoria persistente de las sesiones de trabajo (incluye acuerdos de chats anteriores: pausa del agente ligada al chat de monetización, recordatorio de la API key, estándares del portafolio), verificaciones en vivo contra Supabase, el API local y builds de producción.
- Límites: los chats ajenos a esta línea de trabajo no son legibles directamente; sus acuerdos constan aquí en la medida en que quedaron registrados en la memoria del proyecto. El chat de monetización (task_5348ed05) sigue abierto y sus conclusiones deben incorporarse a las tareas 7-9 al cerrarse.
- Este documento se regenera con docs/audit/build_audit_pdf.py; mantenerlo versionado junto al código.